

Best Python practices -

1. Even for smaller projects, I think having a proper `pyproject.toml` (or at minimum a `setup.py`) and a clear top-level package structure makes a huge difference in usability and maintainability. My rule of thumb is that someone should be able to clone the repo, run `pip install -e .`, and immediately start working without fighting the environment or import paths.
2. I've found doctests to be surprisingly useful, especially for making code more testable and self-documenting. They push me toward writing functions and classes that are easy to instantiate and reason about. I'll often add helper constructors or randomized `@classmethods` so I can quickly generate test objects and validate behavior in small reproducible examples before scaling things up. A style guide I like to use: <https://peps.python.org/pep-0008/>
3. I think Python becomes much more elegant once you fully lean into dunder methods and the language's idioms instead of fighting them. Implementing things like `__repr__`, `__iter__`, `__len__`, comparison operators, or context managers can make custom objects feel natural to use and much easier to debug. In general, I try to write code that feels "Pythonic" rather than forcing patterns from other languages into Python.
4. Another lesson that I will always carry with me from my Software Engineering for Civic Tech class, taught by civic tech pioneer James Turk, is that good Python developers should learn to read the manual before reaching for shortcuts. For me, that means treating official documentation, error messages, source code, and changelogs as primary sources of truth before asking teammates and blasting off a message in Slack "how do I parse JSON in Go?" or overly relying on AI-generated answers. This is not about avoiding collaboration; it is about showing up to collaboration prepared, with a clear understanding of what I tried, what failed, and where I am actually stuck. In Python especially, where libraries move quickly and copy-paste solutions can become outdated fast, this habit builds real debugging judgment.

Tools (highly recommended!) used to visualise code and workflow:

- <https://marketplace.visualstudio.com/items?itemName=CodeViz.codeviz>
- <https://gitdiagram.com>
- <https://marketplace.visualstudio.com/items?itemName=DucPhamNgoc.codevisualizer>